Programando en MIPS

Principio de Computadores

Tutoría Académica nº 2

Introducción: Programando en ensamblador

- Cuando se desea desarrollar un algoritmo normalmente se hace en un lenguaje de alto nivel como C, C++ o Java. Estos programas podrán servir como el pseudocódigo para implementar el algoritmo en ensamblador.
- En esta sesión se explicarán algunos de los elementos básicos que son necesarios para poder realizar los algoritmos de nuestras prácticas:
  - El uso de los registros
  - Las instrucciones enteras
  - Las estructuras de control

Uso de los Registros

- Nuestros programas de alto nivel usan variables para almacenar la información (masa, radio, etc.).
- Estas variables no son más que identificadores que hacen referencia a una posición de memoria donde se almacena el dato correspondiente.
- Ya se ha visto que en ensamblador del MIPS declaramos nuestras variables bajo la directiva `.data`
- Sin embargo, cuando tratamos los datos en MIPS (como en la mayoría de las CPUs), no podemos **operar** directamente con lo que se encuentra en la memoria, sino que los **datos tienen que estar en los registros**.

Uso de los Registros

- Muchas de las variable basta con almacenarlas en un registro.
  - Principalmente las locales que tienen una vida corta
  - No necesitan almacenarse en memoria. No tienen que definirse en la sección `.data`
  - Esta es la solución más eficiente ⇒ tratar de hacerlo siempre.
- Cuando se codifica un algoritmo, es necesario mantener una **tabla de referencia** con los registros que se están usando en el mismo y a qué variable de tu algoritmo en lenguaje de alto nivel hace referencia cada registro.
- Esta información **debe** aparecer en los **comentarios de tu código fuente**.
- Un registro, que en un punto de la ejecución tiene asignada una variable de nuestro programa, más tarde podrá contener otra variable si la primera ya no se necesita (ha salido de hámbito).

Los Registros del MIPS

- MIPS dispone de 32 registros enteros `$0-$31`
  - Cada uno de 32 bits → tamaño palabra (word).
- Para la CPU **todos los registros son equivalentes**, es decir, todos tienen las mismas capacidades y funcionalidades.
- Pero para facilitar la programación, existe un **convenio** (entre los seres humanos) que **asigna funcionalidades específicas** a cada registro.
- Tenemos que respetar el convenio si queremos que nuestros programas puedan ser entendidos y usados de forma correcta por otras personas.
- La persona que programa en ensamblador es responsable de hacer un uso adecuado de los registros.
  - Ni el ensamblador ni la CPU verifica ni controla el uso
- Se definen unos nombres simbólicos (alternativos a `$0-$31`) según la funcionalidad del registro: `$t0`, `$t1`, … `$s0`, `$s1`, … `$sp`, …
  - En nuestros programas de ensamblador **debemos** usar los nombres simbólicos.

Resumen Funcionalidad de Los registros

Num	Nombre	Descripción
0	`$zero`	Constante valor cero
1	`$at`	Temporal reservado para el ensamblador (no utilizar)
2-3	`$v0-$v1`	Valores resultantes de funciones. No se preservan al llamar a subrutinas ni `syscall`.
4-7	`$a0-$a3`	Parámetros de entrada para subrutinas. No se preservan al llamar a subrutinas ni `syscall`.
8-15	`$t0-$t7`	Temporales. No se preservan al llamar a subrutinas ni `syscall`.
16-23	`$s0-$s7`	Valores salvados. Su valor no se modifica cuando llamamos a subrutinas o `syscall`. Una subrutina que los use debe salvar sus valores antes de usarlos, y restaurarlos al salir.
24-25	`$t8-$t9`	Continuación a los $t0-t7. Temporales. No se preservan al llamar a subrutinas ni `syscall`.
26-27	`$k0-$k1`	Reservados para el kernel (manejo de interrupciones)
28	`$gp`	Puntero global. Apunta al medio del bloque de 64K en el segmento de datos estáticos
29	`$sp`	Puntero de pila (stack pointer)
30	`$s8/$fp`	Valor salvado. Puntero de marco. Se preserva a través de llamadas.
31	`$ra`	Dirección de retorno (return address)

Registros Especiales

- `$zero`: su contenido siempre debe estar a valor cero.
  - Se puede usar su valor, pero no se debe modificar.
- `$at`: usado por el ensamblador al implementar pseudoinstrucciones.
  - No debemos utilizarlo al programar
- `$k0-$k1`: Reservados para el kernel para manejo de interrupciones y llamadas al sistema syscall.
  - No debemos utilizarlo al programar
- `$gp` *Puntero global*. Apunta al medio del bloque de 64K en el segmento de datos estáticos. Usado por los compiladores de MIPS para optimizar accesos a memoria
  - No debemos utilizarlo al programar

Registros para variables

- `$s0-$s8`: son registros “salvados” (saved) porque su contenido no se modificará cuando llamemos a una subrutina o llamada al sistema `syscall`.
  - Usarlos para variables de nuestro algoritmo que deban *sobrevivir* a llamadas a subrutina o `syscall`.
  - Cuando **escribamos una subrutina** debemos asegurarnos de **no modificar** su contenido. Si necesitamos utilizar alguno de estos, tendremos que encargarnos de salvarlos al comienzo y restaurarlos antes de regresar (usando la pila).

- `$t0-$t9`: son registros “temporales” y no se garantiza que se conserve su valor cuando llamemos a una subrutina o llamada al sistema `syscall`.
  - Solo debemos usarlos para variables en cuya "vida útil" no hay una llamada a subrutina ni a `syscall`.
  - Cuando **escribamos una subrutina** podemos **modificarlos** con total libertad.

Registros para subrutinas

- `$v0-$v1`: cuando una subrutina (función) tiene que devolver pocos resultados deberá utilizar estos registros para depositarlos.
  - No se garantiza que su valor se conserve al llamar a una subrutina o llamada al sistema `syscall`.
- `$a0-$a3`: cuando una subrutina tenga pocos parámetros de entrada se usarán estos registros para pasar los valores.
  - No se garantiza que su valor se conserve al llamar a una subrutina o llamad al sitema `syscall`.
- `$sp`: puntero de pila (stack pointer). Pila es zona de memoria donde las rutinas almacenan valores temporalmente.
  - Debe manejarse de manera rigurosa según convenio. Los veremos en TA sobre subrutinas (TA6). Uso incorrecto puede provocar errores difíciles de depurar.
- `$ra`: Almacena la dirección a la que debe regresar la subrutina llamada.
  - Veremos su uso cuando se presenten las subrutinas (TA6).

Instrucciones Enteras: Introducción

- El tipo de dato de los registros del MIPS → enteros de 32 bits (word).
- Instrucciones que trabajan con registros de CPU → Instrucciones Enteras.
- Veremos a continuación las más importantes
  - Assemblers, Linkers, and the SPIM Simulator para ver todo el repertorio.
- Los registros utilizados son meros ejemplos, se puede utilizar cualquier otro en su lugar, ya que para la CPU todos los registros son totalmente equivalentes.
- La palabra `etiqueta` igualmente es un ejemplo, un nombre que damos a una zona de memoria y que puede ser cualquier otra.

Instrucciones Movimiento de datos

Instrucción	Equivalente C++	Explicación
`lw $t5,etiqueta`	`$t5 = etiqueta`	*Load word*. Carga en el registro `$t5` el valor entero almacenado en la posición de memoria señalada por `etiqueta`.
`sw $t4,etiqueta`	`etiqueta = $t4`	*Store word*. Guarda el valor entero contenido en el registro `$t4` en la posición de memoria señalada por `etiqueta`
`li $t3,7`	`$t3 = 7`	*Load immediate*. Carga en el registro `$t3` con el literal constante `7`
`la $t6,etiqueta`	`$t6 = &etiqueta`	*Load address*. Carga en el registro `$t6` la dirección de memoria señalada por etiqueta.
`move $t1,$t2`	`$t1 = $t2`	Copia en `$t1` el valor contenido en el registro `$t2`
`mflo $t7`	`$t7 = LO`	*Move from lo*. Mueve el contenido del registro especial `LO` al registro `$t7`
`mfhi $t4`	`$t4 = HI`	*Move from hi*. Mueve el contenido del registro especial `HI` al registro `$t4`

- No es posible movimientos *"memoria a memoria"*
  - Para copiar de una posición de memoria a otra hay que pasar el dato por un registro.

Instrucciones Aritméticas con enteros

Instrucción	Equivalente C++	Explicación
`add $t2,$t3,$t4`	`$t2 = $t3 + $t4`	*Addition*. Suma con signo. En este caso suma los contenidos de los registros `$t3` y `$t4` almacenando el resultado en `$t2`
`addi $t3,$t2,4`	`$t3 = $t2 + 4`	*Addition immediate*. Suma con un valor inmediato (constante). Suma al registro `$t2` el valor inmediato `4` y lo almacena en `$t3`. El inmediato puede ser negativo.
`addi $t4,1`	`$t4 += 1` `$t4 = $t4 + 1`	Ídem que la anterior pero registro origen y destino son el mismo.
`sub $s1,$t2,$s0`	`$s1 = $t2 - $s0`	*Subtract*. Resta. Realiza `$t2-$s0` y almacena el resultado en `$s1`
`mul $s3,$s4,$s5`	`$s3 = $s4 * $s5`	Multiplica `$s4 * $s5` y el resultado lo almacena en `$s3`. Solo los **32 bits menos significativos** resultantes de la multiplicación ⇒ problema con números muy grandes
`mult $t6,$t7`	`(HI,LO)=$t6*$t7`	Multiplica `$t6 * $t7`. El resultado (64 bits) se almacena en registros especiales: en `HI` los 32 bits más significativos y en `LO` los 32 bits menos significativos.
`div $s0,$s1,$s2`	`$s0 = $s1 / $s2`	Divide `$s1/$s2` y el **cociente entero** lo pone en `$s0`
`div $s1,$s2`	`LO = $s1 / $s2` `HI = $s1 % $s2`	Divide `$s1/$s2` colocando: el **cociente** en `LO` y el **resto** en `HI`

- Existe otro conjunto para manejar datos sin signo
  - Si necesitamos manejar enteros muy grándes (cercanos a 2^{31} = 2.147.483.648)

Instrucciones Manipulación de Bits: Introducción

- Realizan operaciones de logica booleana con los bits de los registros de *manera independiente*.
  - El bit 0 del 1er operando se operara con bit 0 del 2º para generar bit 0 del resultado
  - El bit 1 del 1er operando se operara con bit 1 del 2º para generar bit 1 del resultado
  - El bit 2 del 1er operando se operara con bit 2 del 2º para generar bit 2 del resultado
  - ….
  - El bit 31 del 1er operando se operara con bit 31 del 2º para generar bit 31 del resultado
- El resutado de un bit no depende del de al lado
- Registro como contenedor de bits
- Se denominan también "operaciones bit-a-bit"

	31		3	2	1	0
$t1	0	....	0	1	0	1
$t2	0	....	0	0	1	1
$t0	0	....	0	0	0	1

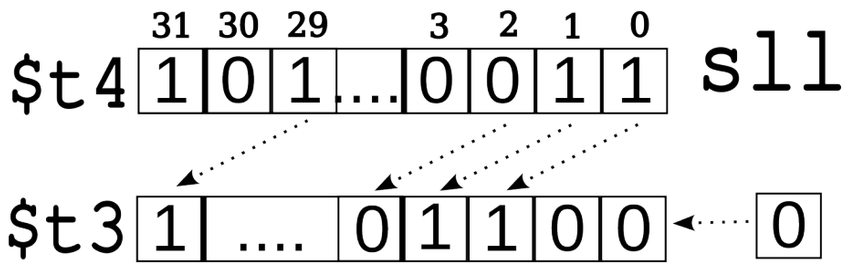
and

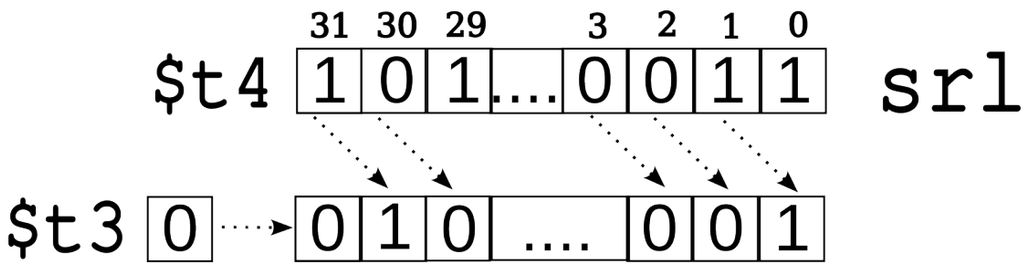
	31		3	2	1	0
$t1	0	....	0	1	0	1
$t2	0	....	0	0	1	1
$t0	0	....	0	1	1	1

or

Instrucciones Manipulación de Bits

Instrucción	Equivalente C++	Explicación
`and $t0,$t1,$t2`	`$t0 = $t1 & $t2`	Realiza la y-bit-a-bit entre registros `$t1` y `$t2` y deposita resultado en `$t0`.
`andi $t0,$t1,7`	`$t0 = $t1 & 7`	Realiza la y-bit-a-bit entre registros `$t1` y la constante `7` y deposita resultado en `$t0`.
`or $t0,$t1,$t2`	`$t0 = $t1 | $t2`	Realiza la o-bit-a-bit entre registros `$t1` y `$t2` y deposita resultado en `$t0`.
`ori $t0,$t1,0xF`	`$t0 = $t1 | 0xF`	Realiza la o-bit-a-bit entre registros `$t1` y la constante (expresada en hexadecimal) `0xF` y deposita resultado en `$t0`.
`xor $t0,$t1,$t2`	`$t0 = $t1 ^ $t2`	Realiza la o-exclusiva-bit-a-bit entre registros `$t1` y `$t2` y deposita resultado en `$t0`.
`xori $t0,$t1,0xD`	`$t0 = $t1 ^ 0xD`	Realiza la o-exclusiva-bit-a-bit entre registros `$t1` y la constante (expresada en hexadecimal) `0xD` y deposita resultado en `$t0`.
`not $t0,$t1`	`$t0 = ~ $t1`	Invierte todos los bits de `$t1` y almacena el resultado en `$t0`
`sll $t3,$t4,2`	`$t3 = $t4 << 2`	*Shift left logical*. Se desplaza 2 bits a la **izquierda** el contenido del registro `$t4`, rellenando con ceros por la derecha. El resultado se almacena en `$t3`.
`srl $t3,$t4,1`	`$t3 = $t4 >> 1`	*Shift right logical*. Se desplaza 1 bits a la **derecha** el contenido del registro `$t4`, rellenando con ceros por la izquierda. El resultado se almacena en `$t3`.





Estructuras de control

- Está demostrado que cualquier algoritmo se puede implemtar mediante la combinación de las siguientes estructuras:
  - estructuras secuenciales → se ejecuta una instrucción detrás de otra
  - estructuras alternativa → según condición se ejecuta unas intrucciones u otras
  - estructuras iterativas → permite la repetición de un conjunto de instrucciones
- La programación estructurada solo permite el uso de esas estructuras. En C y C++ estas estructructuras son:
  - secuenciales → bloque de instrucciones (entre `{ }`)
  - alternativas → If-Else y Switch
  - iterativas → While, Do-While, For

Estructuras de control

- En ensamblador solo existe la estructura secuencial: al terminar una instrucción se ejecuta la siguiente en memoria.
- El resto de estructuras se deben replicar mediante el uso de saltos.
- Los saltos modifican el contador de programa (PC) de manera que la siguiente instrucción a ejecutar no sea la siguiente en memoria.
- La dirección de la instrucción a la que se salta se indica mediante una etiqueta definida en nuestro código fuente
  - unos pocos saltos no usan etiqueta, sino dirección almacenada en registro
- En todas las CPUs existen dos tipos de saltos:
  - Incondicionales → simpre saltan
  - Condicionales → saltan si se cumple condición

Saltos en Mips

- Saltos Incondionales:
  - `j etiqueta  # salta (jump) a línea de código con 'etiqueta:'`
  - `b etiqueta  # salta (branch) a línea de código con 'etiqueta:'`
  - `jr $t3  # salta (jump register) a la dirección de programa contenida en $t3`
- Saltos Condicionales:
  - unos tienen la forma `bCC $t0,$t1,etiqueta`
    - si se cumple la comparación `CC` entre `$t0` y `$t1` se salta a `etiqueta`
    - caso contrario se continúa con siguiente instrucción
  - Las comparaciones dependen si los datos se consideran con signo o sin signo:

Con Signo	Comparación	Sin Signo	Para acordarse
blt	<	bltu	Less Than
ble	<=	bleu	Less or Equal
beq	==	beq	Equal
bne	!=	bne	Not Equal
bge	>=	bgeu	Greater or Equal
bgt	>	bgtu	Greater Than

Saltos en Mips: comparación con cero

- Saltos Condicionales: comparación con cero
  - tienen la forma `bCCz $t0,etiqueta`
    - si se cumple comparación `CC` entre `$t0` y cero se salta a `etiqueta`
    - caso contrario se continúa con siguiente instrucción
  - Las comparaciones posibles son:

	Comparación	Para acordarse
bltz	< 0	Less Than Zero
blez	<= 0	Less or Equal Zero
beqz	== 0	EQual Zero
bnez	!= 0	Not Equal Zero
bgez	>= 0	Greater or Equal Zero
bgtz	> 0	Greater Than Zero

Alternativa If

- En C → se ejecutan instrucciones si se cumple condición
- En ensamblador → **SALTAMOS** si se cumple condición
- Queremos **mantener orden de las instrucciones y de los operandos** de la comparación → hay dos alternativas:
  - SIN invertir la comparación

```
if ($s0 != $s1) {
    // si se cumple → parte then
    $s3 = $s1;
}
// después if
```

```
if:
    bne     $s0,$s1,if_then
    b       if_fin
if_then:
    # si se cumple condición → then
    move    $s3,$s1
if_fin:
    # después if
```

  - INVERTIR la comparación y evitamos el salto extra

```
if ($s0 != $s1) {
    // si se condición → parte then
    $s3 = $s1;
}
// después if
```

```
if:
    beq     $s0,$s1,if_fin
    # si se cumple condición → then
    move    $s3,$s1
if_fin:
    # después if
```

Alternativa If-Else

- Si existe parte `else` ⇒ ensamblador necesario salto incondicional al final parte **`then`** para evitar ejecutar parte **`else`**.
  - SIN invertir comparación:

```
if ($s0 != $s1) {
    // si se cumple → parte then
    $s3 = $s1;
} else {
    // si no se cumple condición
    $s3 = $s0;
}
// después if
```

```
if:
    bne     $s0,$s1,if_then
    b       if_else
if_then:
    # si se cumple condición → then
    move    $s3,$s1
    b       if_fin # evitar parte else
if_else:
    # si no se cumple condición
    move    $s3,$s0
if_fin:
    # después if
```

  - INVERTIR la comparación y evitamos el salto extra

```
if ($s0 != $s1) {
    // si se cumple → parte then
    $s3 = $s1;
} else {
    // si no se cumple condición
    $s3 = $s0;
}
// después if
```

```
if:
    beq     $s0,$s1,if_else
    # si se cumple condición → then
    move    $s3,$s1
    b       if_fin # evitar parte else
if_else:
    # si no se cumple condición
    move    $s3,$s0
if_fin:
    # después if
```

ALTERNATIVA: SWITCH

- Alternativa Switch
  - según algoritmo que ejecuta

```
// leo eleccion por consola
std::cin >> eleccion;

switch(eleccion) {
case 2:
    valor *= 2;
    break; // nos saca swtich
case 1:
    valor /= 2;
    break; // nos saca swtich
case 7:
    valor += 5;

case 4:
    valor *= 3;
    break; // nos saca swtich
default:
    valor++;
}
// después switch
```

```
# eleccion → $t0
# valor → $s0

    # leo eleccion por consola
    li      $v0,5
    syscall # lo leido en $v0
    move    $t0,$v0
switch:
    beq     $t0,2,case_2
    beq     $t0,1,case_1
    beq     $t0,7,case_7
    beq     $t0,4,case_4
    b       default

case_2:
    mul     $s0,$s0,2
    b       switch_fin
case_1:
    div     $s0,$s0,2
    b       switch_fin
case_7:
    addi    $s0,$s0,5

case_4:
    mul     $s0,$s0,3
    b       switch_fin
default:
    addi    $s0,$s0,1

switch_fin:
// después switch
```

Iterativa While

- Entra y permanece `while` mientras se cumple condición
- En ensamblador → **SALTAMOS** si se cumple condición

```
suma = 0;
std::cin >> numero;

while (numero > 0) {
    // interior while
    suma += numero;
    std::cin >> numero;
}
// después while
```

```
# solución SIN invertir comparación
# suma → $s0
# numero → $v0
    move    $s0,$zero

    # leo entero por consola
    li      $v0,5
    syscall # entero en $v0

while:
    #salimos si numero <= 0
    bgtz    $v0,while_dentro
    b       while_fin
while_dentro:
    # interior de while
    add     $s0,$s0,$v0
    # leo entero por consola
    li      $v0,5
    syscall # entero en $v0

    b       while # vuelvo condición
while_fin:
    # después while
```

```
# solución INVERTIR comparación
# suma → $s0
# numero → $v0
    move    $s0,$zero

    # leo entero por consola
    li      $v0,5
    syscall # entero en $v0

while:
    #salimos si numero <= 0
    blez    $v0,while_fin

    # interior del while
    add     $s0,$s0,$v0
    # leo entero por consola
    li      $v0,5
    syscall # entero en $v0

    b       while # vuelvo condición
while_fin:
    # después while
```

Iterativa While: condición al final

- Otra opción poner condición al final
  - se CAMBIA orden de las instrucciones
  - SIN invertir comparación

```
suma = 0;
std::cin >> numero;

while (numero > 0) {
    // interior while
    suma += numero;
    std::cin >> numero;
}
// después while
```

```
# suma → $s0
# numero → $v0
    move    $s0,$zero

    # leo entero por consola
    li      $v0,5
    syscall # entero en $v0

    # Saltamos a condición del while
    b       while_condicion
while:
    # interior del while
    add     $s0,$s0,$v0
    # leo entero por consola
    li      $v0,5
    syscall # entero en $v0

while_condicion:
    #repetimos si numero > 0
    bgtz    $v0,while
while_fin:
    # después while
```

ITERATIVA FOR

- En C un `for` es totalmente equivalente a un `while`
  - usar cuando se sabe número de iteraciones al comenzar → bucles con contador

```
// sumar 100 enteros
suma = 0;

for(int i=0; i<=100; i++) {
    suma += i;
}
// después for
```

```
// sumar 100 enteros
suma = 0;

int i=0; // inicilación
while(i<=100){ // condición
    suma += i;
    i++; // actualización
}
// después for
```

```
# solución SIN invertir la comparación
# suma → $s0
# i → $t0 no hay llamadas syscall ni funciones
    move    $s0,$zero

    move    $t0,$zero #inicilización i=0
for:
    #condición → entramos si i<=100
    ble     $t0,100,for_dentro
    b       for_fin
for_dentro:
    add     $s0,$s0,$t0

    addi    $t0,$t0,1   # actualización i++
    b       for # vuelta condición
for_fin:
    # después for
```

```
# Solución INVIRTIENDO la comparación
# suma → $s0
# i → $t0 no hay llamadas syscall ni funciones
    move    $s0,$zero

    move    $t0,$zero #inicilización i=0
for:
    #condición invertida → salimos si i> 100
    bgt     $t0,100,for_fin
    add     $s0,$s0,$t0

    addi    $t0,$t0,1   # actualización i++
    b       for # vuelta condición
for_fin:
    # después for
```

Iterativa Do-While

- No hay que invertir la condición

```
suma = 0;
maximo = 100;

do {
    std::cin >> numero;
    suma += numero;
} while (suma < maximo);
// después do-while
```

```
# suma → $s0
# maximo → $s1
# numero → $v0
    move    $s0,$zero
    li      $s1,100

doWhile:

    # leo entero por consola
    li      $v0,5
    syscall # entero en $v0

    add     $s0,$s0,$v0

    #volvemos si numero > 0
    blt     $s0,$s1,doWhile
doWhileFin:
    # después doWhile
```

Condiciones más complejas

- En ocasiones las condiciones están compuestas de varias comparaciones
- Se pueden usar registros para almacenar el valor booleano del resultado de una comparación
  - Instruciones SET de la forma `sCC $t0,$t1,$t2`
    - Si se cumple comparación `CC` entre `$t1` y `$t2` se pone `$t0` a `1`
    - caso contrario `$t0` se pone a `0`
  - Como en los saltos, las comparaciones dependen si datos con signo o sin signo:

Con Signo	Comparación	Sin Signo	Para acordarse
slt	<	sltu	Less Than
sle	<=	sleu	Less or Equal
seq	==	seq	Equal
sne	!=	sne	Not Equal
sge	>=	sgeu	Greater or Equal
sgt	>	sgtu	Greater Than

CONDICIONES MÁS COMPLEJAS: OPERACIONES BIT-A-BIT

- Los valores booleanos se pueden combinar utlilizando operaciones booleanas bit a bit:
  - `and $t0,$t1,$t2` `or $t0,$t1,$t2` `not $t0,$t1`
  - Operan con los 32 bits → para las condiciones solo nos interesa el bit 0
    - resto de bits del registro a 0

```
if( (($s1>$s2) && ($s1<=$s3)) || ($s4<0)) {
    // se cumplió la condición
    $s1++;
}
// después if
```

```
if:
    sgt     $t0,$s1,$s2   # $t0 = ($s1>$s2)
    sle     $t1,$s1,$s3   # $t1 = ($s1<=$s3)
    and     $t0,$t0,$t1   # $t0 = ($s1>$s2) && ($s1<=$s3)

    slt     $t2,$s4,$zero  # $t2 = ($s4<0)

    or      $t0,$t0,$t2    # $t0 = (($s1>$s2) && ($s1<=$s3)) || ($s4<0)

    beqz    $t0,if_fin     # si $t0 es 0 ⇒ NO se cumplío condición

    # se cumplió la condición
    addi    $s1,$s1,1

if_fin:
    # después if
```

Condiciones más complejas: Ejemplo otra forma

- También se puede resolver con saltos condicionales bien elegidos

```
if( (($s1>$s2) && ($s1<=$s3)) || ($s4<0)) {
    // se cumplió la condición
    $s1++;
}
// después if
```

```
# posible solución
# SIN invertir condiciones
if:
    bgt     $s1,$s2,if_mirarS3
    b       if_mirarS4
if_mirarS3:
    ble     $s1,$s3,if_then
if_mirarS4:
    bltz    $s4,if_then
    b       if_fin
if_fhen:
    # se cumplió la condición
    addi    $s1,$s1,1

if_fin:
    # después if
```

```
# posible solución eligiendo
# condición más adecuada
if:
    ble     $s1,$s2,if_mirarS4
    ble     $s1,$s3,if_then
if_mirarS4:
    bgez    $s4,if_fin

if_then:
    # se cumplió la condición
    addi    $s1,$s1,1

if_fin:
    # después if
```